

CloudCompass Builder — Lab 3 Report

Course: Cloud Solutions Architectures **Group:** Group 2 **Members:** Agustin Gentil, Boumediene Rayane Mazari, Diego Alfaro Gómez, Renato González Huamán, Sebastián Otegui Gómez **Submission date:** 2026-06-28

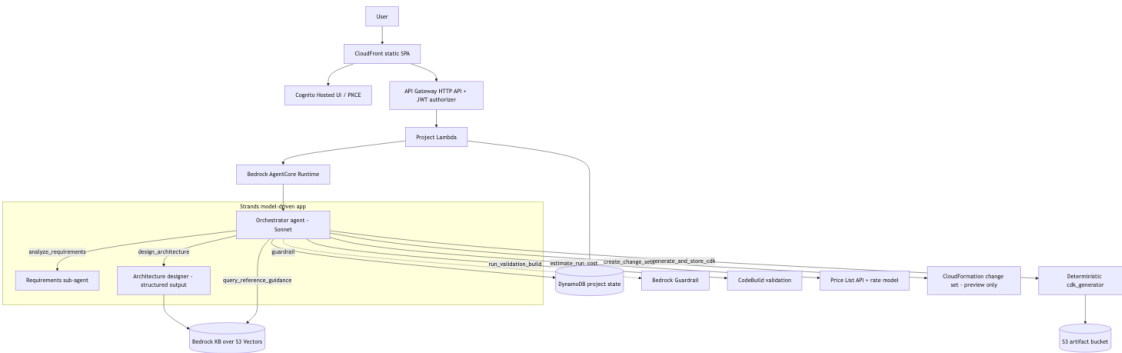
Status: *deployed and demoed end-to-end* in AWS account `777170294579` (*us-east-1*). All 8 stacks are live; the AgentCore runtime is `READY` ; the demo prompt runs the full model-driven workflow to `CHANGE_SET_READY` . See §10 for the as-deployed evidence.

1. Use Case

CloudCompass Builder turns a natural-language AWS infrastructure request into a **governed, reviewable deployment package**: a validated Python CDK project, a security check, a per-run cost estimate, and a CloudFormation **change set** that a human reviews before anything is created. The demo vertical is an online bakery needing a static website, customer login, an order API, order storage, email receipts, logs, and a monthly budget under \$200.

The agentic layer adds value over a template picker because a **Bedrock model reasons** about the request: it extracts requirements, retrieves approved guidance (RAG), and *designs* an architecture spec from the prompt — rather than returning a fixed template. Determinism is reserved for the one step that must not drift: the CDK *code generation*, which uses a tested Jinja2 renderer so the emitted project is reproducible and unit-testable. Reasoning is the model's; code generation is deterministic.

2. Architecture and Workflow



AWS-native building blocks, organized as 8 CDK stacks:

Layer	Services
Frontend + auth	S3 private bucket, CloudFront (Origin Access Control), Cognito Hosted UI (PKCE)
API entry	API Gateway HTTP API (v2) with a Cognito JWT authorizer
Entry compute	Lambda (projects, agent_invoker, upload_url) on ARM64, X-Ray active
Agentic runtime	Strands Agents on Amazon Bedrock AgentCore Runtime (ARM64 container)
Foundation model	Bedrock — Claude Sonnet-class inference profile (orchestration) + optional Haiku (cheap subtasks)

Safety	Bedrock Guardrail (prompt-injection + content filters) on every model call
Knowledge / RAG	Bedrock Knowledge Base over S3 Vectors , seeded guidance corpus
Generation	cdk_generator — deterministic Jinja2 Python-CDK renderer
State + artifacts	DynamoDB single-table (project state), S3 (generated CDK zip, manifest, preview template)
Validation + preview	CodeBuild (synth/compile the generated project) + CloudFormation change set
Observability	CloudWatch Logs (1-month retention), X-Ray, CloudWatch dashboard, AgentCore platform telemetry

Execution flow:

1. User signs in through the Cognito Hosted UI (authorization-code + PKCE); the SPA stores the ID token.
2. Browser submits `POST /projects { prompt, region }`.
3. API Gateway validates the Cognito JWT; the project Lambda derives `user_id` from the JWT `sub` (never from the body), writes `RECEIVED` to DynamoDB, and invokes the AgentCore runtime.
4. The **Strands orchestrator (Sonnet)** drives the workflow, calling tools and specialist sub-agents:
 - `analyze_requirements` → a requirements sub-agent extracts the workload;
 - `query_reference_guidance` → retrieves approved guidance from the Bedrock Knowledge Base (S3 Vectors), with a curated fallback;
 - `design_architecture` → an architecture sub-agent emits a **schema-validated** spec via structured output (retry-on-invalid);
 - `generate_and_store_cdk` → the deterministic generator renders the CDK project, a lightweight security check runs, and the artifact is zipped to S3;
 - `run_validation_build` → starts CodeBuild on the artifact;
 - `estimate_run_cost` → computes the per-run cost;
 - `create_change_set` → creates a CloudFormation change set (preview only).
5. Each tool persists its result to DynamoDB; the run ends at `CHANGE_SET_READY`.
6. The UI polls `GET /projects/{project_id}` and displays the architecture summary, artifact download link, validation result, cost estimate, and change-set ARN.

A second ingestion flow exists: `POST /upload-url` mints a per-user presigned S3 PUT URL (key namespaced to `users/{sub}/...`).

3. Technical Decisions

Decision	Choice	Reason
Agent topology	Model-driven orchestrator + specialist sub-agents, no deterministic fallback	The model genuinely drives requirements + architecture; satisfies the agentic rubric, not just a pipeline with agents bolted on
Foundation model	Strands' Sonnet-class inference profile (*.anthropic.claude-sonnet-4-6); Haiku for cheap subtasks via <code>BEDROCK_FAST_MODEL_ID</code>	Sonnet for tool-use/instruction-following; Haiku to cut cost on extraction

Runtime	Bedrock AgentCore Runtime (no L2 construct yet → control-plane custom resource with <code>on_update</code>)	Managed agent hosting required by the lab; <code>on_update</code> rolls new images to the runtime in place
Codegen	Deterministic Jinja2 renderer (not LLM-emitted CDK)	Repeatable, snapshot-testable output; no syntax drift; the model selects services, the renderer emits code
RAG	Bedrock Knowledge Base over S3 Vectors	Real retrieval grounding; curated fallback keeps the demo resilient before/around ingestion
Safety	Bedrock Guardrail (PROMPT_ATTACK + content filters) attached to the model	Prompt-injection defense on a tool-calling agent
API	HTTP API v2 + Cognito JWT authorizer	Lower cost/latency than REST; native JWT support
Frontend	Private S3 behind CloudFront OAC	No public origin bucket; CDN delivery
Validation	CodeBuild (no-source, artifact URI at runtime)	Isolated <code>pip install + compileall + cdk synth</code> of generated artifacts
Deployment boundary	CloudFormation change set only	Human review before any real resource is created
AgentCore Gateway	Out of scope (deliberate)	The orchestrator uses in-process Strands <code>@tools</code> (simpler, lower-latency, fully governed). Gateway exposes tools over MCP to <i>external</i> agents — not needed for a single-agent MVP

4. Service Configuration

Service	Configuration
S3 (artifact / frontend / corpus)	SSE-S3, Block Public Access (all four), TLS-only, versioned; frontend + corpus served privately
DynamoDB	pk/sk single table, on-demand, AWS-managed KMS, PITR, TTL (<code>#ttl</code> , 14-day)
CloudFront	Private S3 origin via OAC, redirect-to-HTTPS, SPA 403/404 → <code>index.html</code>
Cognito	Email sign-in, hosted domain, authorization-code + PKCE, strong password policy
API Gateway	Cognito JWT authorizer on <code>/projects</code> , <code>/projects/{id}</code> , <code>/invoke</code> , <code>/upload-url</code>
Lambda	Python 3.13, ARM64, X-Ray active, 1-month log retention
AgentCore runtime	ARM64 image (ECR), PUBLIC network, HTTP protocol, env-injected resource names + guardrail id; execution role scoped (model/profile, Retrieve, DDB, S3, CodeBuild, scoped CFN, ApplyGuardrail)

Bedrock Guardrail	PROMPT_ATTACK (HIGH/input) + HATE/VIOLENCE/MISCONDUCT filters; DRAFT version
Knowledge Base	VECTOR type over S3 Vectors (Titan Text Embeddings V2, 1024-dim, cosine); S3 data source over the seeded corpus
CodeBuild	No-source, python -m compileall + cdk synth of the generated project
CloudFormation	Change-set creation only from the agent role; execution out of scope

5. Agentic Implementation

The AgentCore container runs a **genuinely model-driven** Strands application — the previous iteration defined agents but never called the model; this one does.

- **Orchestrator (Sonnet-class)** receives the prompt and drives the workflow by calling tools, in order, never fabricating results.
- **Requirements analyst (fast/Haiku-class)** is invoked as the `analyze_requirements` tool to extract workload/budget/missing-info.
- **Architecture designer (Sonnet-class)** is invoked as the `design_architecture` tool and returns a spec via **Pydantic structured output**, which is coerced to the generator's JSON-Schema constraints and validated, retrying up to 3x on invalid output — so the spec is prompt-specific *and* schema-valid.

Tools are **bound per request** to `user_id / project_id / region`, so the model never manages identity, and each mechanical tool persists its own result to DynamoDB. The supported service catalog (the architecture designer must use only these) is: `s3_bucket`, `cloudfront_site`, `cognito_user_pool`, `lambda_api`, `dynamodb_table`, `ses_email`.

In the live demo the agent designed all six services from the bakery prompt (website → CloudFront+S3, login → Cognito, API → Lambda+HTTP API, orders → DynamoDB, receipts → SES, assets/logs → S3) — not a hardcoded template.

6. Security

- **Identity from JWT, not the body.** `user_id` is the Cognito `sub`; records are keyed `PK=USER#{sub}`, `SK=PROJECT#{id}`.
- **Least privilege, verified by test.** A unit test asserts **no bare * IAM action across all 8 stacks**. Each component (Lambda, runtime, CodeBuild, CloudFormation, KB) uses a separate role.
- **Scoped model access.** The runtime role allows `bedrock:InvokeModel` only on `anthropic.claude-*` foundation models + inference profiles (required for the Sonnet profile), and `bedrock:Retrieve` only on the project KB.
- **Governed deployment.** The agent role can **create/describe** change sets but **not execute** them (`cloudformation:ExecuteChangeSet` is absent — asserted by test).
- **Prompt-injection defense.** A Bedrock Guardrail (PROMPT_ATTACK + content filters) is attached to every model call.
- **Buckets** block public access and enforce TLS; **DynamoDB** has PITR + encryption; generated CDK defaults are private/encrypted.
- **Documented broad grant:** the *deploy-time* custom-resource provider role uses `bedrock-agentcore:*` because `CreateAgentRuntime` fans out to many undocumented sub-operations

(endpoint, service-linked role, workload identity). This is confined to the provisioning Lambda; the runtime's own execution role stays tightly scoped.

7. Cost Estimate

Meaningful unit: one infrastructure-generation run (prompt → requirements → RAG → architecture → CDK generation → validation → change set).

The estimator **computes** cost from the run's usage and transparent unit rates (us-east-1, 2026); it also confirms the CodeBuild rate from the AWS Price List API at runtime (falling back to the curated rate offline).

Component	Driver	Unit rate
Bedrock model inference	input + output tokens	\$0.003 / 1K in, \$0.015 / 1K out (Sonnet-class)
CodeBuild validation	small build-minutes	\$0.005 / min
Lambda + API Gateway + DynamoDB + S3 + logs	per run	~\$0.01 fixed overhead

Worked examples (variable cost per run):

Scenario	Tokens (in/out)	Build min	Computed cost
Light run (demo self-estimate)	2.1K / 0.95K	3.5	\$0.048
Typical multi-agent run	~20K / 8K	3	\$0.21
Heavy run	40K / 30K	5	\$0.61

So **~\$0.05–0.60 per run**, typically ~\$0.20, dominated by output tokens. The demo run reported **\$0.0481** (verified live).

Fixed / low-baseline monthly costs (the CloudCompass *platform*, idle):

Component	Notes
ECR image storage	~\$0.10 (one ARM64 image)
S3 / DynamoDB storage	Negligible (small artifacts + metadata)
CloudWatch Logs	Bounded by 1-month retention
AgentCore runtime	Billed per active second; ~\$0 idle
KB / S3 Vectors	Small for a 3-document corpus
CloudFront / Cognito	Request-driven; ~\$0 at demo scale

The cost of *running* the generated bakery infrastructure (if its change set were executed) is separate and out of MVP scope — CloudCompass stops at the preview.

8. Scalability and Production Readiness

Every tier is a managed service with a natural scale-out boundary: CloudFront (UI), API Gateway + Lambda (entry), AgentCore (agent), DynamoDB on-demand (state), S3 (artifacts), CodeBuild (isolated validation), Bedrock KB (retrieval). Production hardening would add: a queue/Step Functions for long validations, stricter CORS origins, published Guardrail + KB versions, a wider generated service catalog, automated KB re-ingestion on corpus change, and an explicit human-approval workflow for change-set *execution*.

9. CDK Code Reference

Account-agnostic Python CDK, 8 stacks (`cdk deploy --all`):

1. `CloudCompass-Storage` — artifact bucket, project table (TTL), EventBridge bus.
2. `CloudCompass-FrontendHosting` — private S3 frontend bucket + CloudFront OAC.
3. `CloudCompass-Auth` — Cognito user pool, app client, hosted domain.
4. `CloudCompass-Validation` — CodeBuild project + scoped CloudFormation execution role.
5. `CloudCompass-Knowledge` — S3 Vectors bucket + index, Bedrock Knowledge Base, S3 data source, seeded corpus.
6. `CloudCompass-Agent` — ECR image (ARM64 Strands agent), guardrail, execution role, AgentCore runtime (custom resource with `on_update`).
7. `CloudCompass-API` — authenticated HTTP API, Lambdas, observability dashboard.
8. `CloudCompass-FrontendDeployment` — static UI + generated `config.json` .

The user-facing **generated** CDK lives under `cdk_generator/` (a product feature) and is separate from `cdk_app/` , which deploys CloudCompass itself. Test suite: 34 passed / 2 skipped; both generated example projects pass real `cdk synth` .

Runs entirely on AWS. At runtime nothing executes locally — the agent runs on Bedrock AgentCore, the API on API Gateway + Lambda, state in DynamoDB, etc. Docker is a *build-time* tool only: AgentCore runs container images, so `cdk deploy` builds the agent image once and pushes it to ECR (AWS), where AgentCore runs it (analogous to needing Node.js for the CDK CLI). Reviewing the code, running the tests, and `cdk synth` (inspecting all 8 CloudFormation templates) need **no Docker**; only an actual `cdk deploy` does.

10. As-Deployed Evidence and Lessons

Deployed (us-east-1, account 777170294579): all 8 stacks `CREATE_COMPLETE` ; runtime `cloudcompass_c89e0089-...` `READY` ; KB `MAA437H2XF` ingested (3/3 docs); API `https://bh83je4x1h.execute-api.us-east-1.amazonaws.com/` ; site `https://d32nsp1wucrr32.cloudfront.net/` . A demo invocation produced a real generated CDK zip in S3, a passing security check, a `$0.0481` cost estimate, and a CloudFormation change set (`CREATE_COMPLETE` , `AVAILABLE` , **not executed**) planning the bakery's Cognito/SES/S3/Lambda/DynamoDB/CloudFront resources.

Two defects were found only by the live deploy (neither caught by synth or unit tests, both fixed): (1) the AgentCore provisioning role needed endpoint + service-linked-role + workload-identity permissions; (2) `ttl` is a DynamoDB reserved keyword, so `save_project_state` returned 500 on every invocation until escaped as `#ttl` . This is why an end-to-end deploy — not just `cdk synth` — is part of the deliverable.

Limitations / assumptions: single-region; AgentCore PUBLIC network mode; KB retrieval returns matches only after an ingestion job runs (curated fallback until then); the account must submit the Bedrock Anthropic use-case form once; the change-set preview is a conservative representation (the generated CDK zip is the authoritative artifact); CORS is permissive for the demo.